What We Actually Built — A Plain-Language Walkthrough

This is the "explain it to me like I want to actually understand it" version. The professional document says *what* was done in formal language for other people to read. This one is for you — walking through *why* each decision happened, in the order it happened, so the whole project makes sense as a story rather than a list of facts.

The big picture first

Your job on this project was to build the "brain" of Shadow Kedi — the part that looks at everything happening on a company's computers (logins, file transfers, USB drives, web browsing, app installs) and decides: *is this normal, or is this a real security problem?*

You didn't have real company data to practice on yet (the actual Wazuh monitoring tool isn't hooked up), so the plan was always: build and test everything on realistic fake/practice data first, then swap in real data later without having to rebuild everything.

By the end, your system correctly catches **98.4% of real security incidents**, with very few false alarms. That number didn't come easily — it came from a long chain of "this isn't working well, let's figure out *why*, fix that specific thing, and check if it actually helped." That chain is what this document walks through.

Part 1: Getting data to practice on

You needed fake-but-realistic data to build and test against. You ended up with two very different data sources:

Your own made-up company data ("CASB") — you built a generator that creates a fake company with 220 employees, and simulates 90 days of their computer activity, including some employees doing "bad" things like using unauthorized AI tools or trying to steal data before quitting. You control every detail of this one because you built it.

A real, famous research dataset ("CERT") — this one is genuinely real. Researchers at Carnegie Mellon built a dataset simulating 1,000 employees over about 17 months, and — this is the important part — they actually had people play the role of "bad actors" doing real insider-threat scenarios (stealing data before quitting, planting a keylogger, etc.). It's 32.7 million individual actions. This one is licensed, meaning you can use it but you can't publish the raw data itself online — only your code that processes it.

You originally tried a different dataset (called NSL-KDD) first, but realized it was just raw internet traffic data with no connection to "a person doing something suspicious at work" — so you dropped it and used CERT instead, which was a much better fit.

Part 2: The computer almost crashed (memory problem)

Early on, you tried to process all 32.7 million CERT events at once, and your computer's memory filled up to 97% and had to be force-killed. The problem: the code was trying to hold *everything* in memory at once before saving any of it.

The fix: instead of "load everything, then save," the code was rewritten to "process one piece, save it immediately, move to the next piece" — like reading a book one page at a time instead of trying to hold the entire book open in your hands at once. After this fix, the same 32.7-million-event job ran fine without ever stressing your computer's memory.

Part 3: Teaching the system what "normal" looks like

To catch something *unusual*, the system first needs to know what's *usual* for each person. This is called a "baseline" — like knowing someone's normal walking pace so you can tell if they're suddenly running.

Here's a subtle bug you caught: because there were SO many more "normal" events than "bad" events in the CERT data, you had to only keep a small sample of the normal ones (about 1 in 22) to make the file sizes manageable — but you kept *all* of the bad ones. That created a sneaky problem: for someone who was actually doing bad things, their "normal baseline" ended up being built mostly from their OWN bad behavior, since so few of their normal actions survived the sampling. It's like trying to figure out someone's "normal mood" by only interviewing them on their worst days — you'd end up thinking their worst days *were* normal for them.

The fix: you changed the code so baselines are only ever built from confirmed-normal behavior, never from anyone's bad days, even their own.

Interesting twist: fixing this bug barely changed the results at all. That told you the REAL problem was something else entirely — which leads to the next part.

Part 4: The big discovery — why looking at single actions doesn't work

This is probably the most important thing you figured out in the whole project.

You were trying to judge each individual action (a single login, a single web page visit) as "suspicious" or "not suspicious." But here's the thing about the CERT dataset: when a researcher labeled someone as a "bad actor," they labeled their ENTIRE multi-day period as bad — not just the specific moment they did something wrong.

Think about it this way: if someone plans to steal company data and does it on a Tuesday afternoon, they still log in normally every morning, browse the web normally most of the day, and only do ONE genuinely suspicious thing during that whole window. But the dataset labels *everything* they did that week as "malicious" — including their totally normal Monday morning email check.

You proved this directly: you compared how often "bad" logins happened after-hours versus "normal" logins, and the difference was tiny (18% vs 13% — barely different at all). That confirmed it: **most of what's labeled "bad" looks completely ordinary when you examine it one action at a time**, because it genuinely IS ordinary — it just happened to occur during a labeled bad stretch.

This is why single-action detection could never work well — you weren't missing a clever trick, the information you needed just wasn't there at that zoomed-in level.

Part 5: The fix — zoom out to "per person, per day"

Once you knew the problem, the fix became obvious: instead of judging each action alone, **add up everything a person did in one day** and judge the whole day as a pattern.

Why does this work? Because on the day someone actually does something bad, that day's *pattern* looks different — maybe an unusual spike in file transfers, or connecting a USB drive at an odd hour, or a burst of unusual browsing. Even if 90% of that day was totally normal, the *cumulative* picture for that specific day stands out, even though no single action in isolation would have.

This one change was the single biggest improvement in the entire project. Detection performance for the real CERT data went from "barely better than a coin flip" to "catching over 97% of real incidents." Everything clicked into place once you looked at the right zoom level.

Part 6: Getting the "how risky is this?" score to actually make sense

Once the detection was working, you needed to turn "this looks suspicious" into a clean 0–100 risk score that a human could glance at and understand — matching what the project blueprint asked for.

This took two separate fixes:

Fix 1 — the raw math didn't naturally spread across 0–100. The formula you were using tended to produce small numbers clustered near zero, not spread out nicely. You fixed this by ranking every day against every other day (percentile-style, like "this day is in the top 3% most unusual") rather than trying to force the raw number to mean something on its own.

Fix 2 — ranking-based alerts flagged way too much stuff. Once you had percentile ranks, you initially used them to also decide WHO gets flagged as high-risk — but that backfired badly, because a percentile rank always flags a fixed percentage of days no matter what, even if nothing that risky actually happened that month. You fixed this by separating two different questions: "what number do we show on the dashboard" (fine to use percentile

ranking for) versus "should this trigger an actual alert" (needs to be based on how *confident* the trained model genuinely is, not a fixed percentage).

Fix 3 — one company's data was drowning out the other's. Your CERT data has about 17 times more flagged incidents than your CASB data. When you used one shared "alert threshold" for both, it ended up being tuned mostly for CERT's patterns, and CASB's real incidents were slipping through because they just don't hit the same numbers CERT does. You fixed this by giving each data source its own separately-tuned threshold, found by testing many different cutoff points and seeing which one worked best for each source specifically.

Part 7: Even your genetic algorithm had the same "one source drowns out the other" bug

You built a genetic algorithm (a search technique that "evolves" better solutions over many generations, like breeding) to figure out which of your ~38 tracked details actually matter most, and to tune the model's settings.

The genetic algorithm's job was to try lots of combinations and reward whichever one performed best. But "best" was being measured as one combined score across BOTH data sources — and because CERT has so much more data, the search kept finding settings that helped CERT while quietly hurting CASB. You caught this directly: one setting it landed on made CASB's accuracy get noticeably worse.

The fix: instead of judging "best" by one combined number, you changed it to judge by the *average of each source's own individual score*. That way, the genetic algorithm can't win by sacrificing one source to help the other — it has to actually do well on both to score well overall.

This fix didn't just prevent the CASB damage — it accidentally found your BEST model of the whole project. The new best settings improved BOTH data sources at the same time, while also using fewer of the 38 details (meaning it's a simpler, easier-to-explain model too).

Part 8: A real decision you made on purpose — not just chasing the highest number

Near the end, you had a choice: your simpler "decision tree" model was slightly less accurate than a more complex "random forest" model (a random forest is basically 200 decision trees averaged together). You tested what would happen if you used the more complex model just for your CASB data.

The complex model *would* have caught more real incidents. But it also would have created about **10 times more false alarms** to do it — meaning IT staff would need to investigate roughly 43 more false alerts just to catch 13 more real ones.

You chose to keep the simpler model anyway. Here's the reasoning: for a small IT team, the real bottleneck isn't "can we technically catch more stuff" — it's "how much can a human actually review." A system that cries wolf 10x more often, even if it's technically more sensitive, is often *worse* in practice. Plus, a single decision tree can be explained in plain language ("flagged because of X, Y, Z"), while a 200-tree average basically can't — and being explainable, rather than a black box, was one of the whole point of this project.

This wasn't the model with the best number on paper. It was the better real-world choice, and you can explain exactly why.

Part 9: Turning "this is risky" into "here's what to actually do"

A risk score alone isn't that useful to a small business owner — they need to know what ACTION to take. You built two systems for this:

System 1 — sourced recommendations. A teammate researched real security practices (backed by actual studies and advisories, not just opinions) for 9 categories of "shadow IT" behavior. You connected your risk detection to this research so that when something gets flagged, the system can say "here's a real, cited fix for this specific problem" — not just "something's wrong, good luck."

While doing this, you noticed a gap: one whole category of risky behavior (people installing unauthorized remote-access software like AnyDesk or TeamViewer) had no matching research section. So you researched it yourself and wrote a new section, citing a real government cybersecurity advisory — and in the process, you caught and corrected a mistake in a vendor's blog post that had mischaracterized which advisory said what.

System 2 — turning risk scores into action tiers. A teammate separately designed a system of 6 action levels (from "just log it, no action needed" up to "escalate to security immediately"). You built the connector between your risk scores and their action-tier system — and while testing it carefully (not just trusting it worked), you found and fixed two real logic bugs:

- A very low-risk event with a lot of "everyone's doing it" pattern was incorrectly getting escalated when it shouldn't have.
- A rule meant to say "this matches a known-bad thing, block it" was accidentally overriding a DIFFERENT, smarter rule that said "this is so widespread it's probably a legitimate business need — offer people a proper approved tool instead of just blocking them." That second rule matters a lot for AI tools specifically, since the research your teammate found shows "offer a proper alternative" works way better than "just block it" for that category.

You fixed both bugs, then wrote 27 automated tests that check this logic will never break again the same way — the first real test suite anywhere in this project.

Part 10: Where things stand now

Your whole system, in one line: raw computer activity → cleaned up and standardized → summarized per person per day → scored for how unusual it is → turned into a 0–100 risk number → turned into a specific recommended action, with real sources cited.

Every improvement along the way is something you can point to and explain: here's the problem, here's how I found it, here's the fix, here's the proof it worked. That's a genuinely strong thing to be able to say about a project — it's not just "it works," it's "it works, and I know exactly why."

What's still left, if you want to keep going:

- Real tests for the rest of the pipeline (you've only built the full test safety-net for one piece so far)
- Waiting on your teammates for a few missing pieces (real Wazuh data samples, how the backend wants to receive your results, what the dashboard needs to display)
- Opening the actual "merge this into the main project" request on GitHub (your code is pushed and safely saved, just not yet merged into the main branch)